

# AVAL

Pagamento agêntico com mandato verificável

O LLM propõe.  
O núcleo determinístico dispõe.  
O modelo nunca está no  
**caminho de confiança.**

A ESCADA DE AVALIAÇÃO

1

2

3

mandate\_revoked – para aqui

4

5

6

7

8

9

10

11

12

13

14

15

16

Treze degraus **nunca consultados** — e é a ausência deles que prova que uma revogação não pode ser contornada por uma compra pequena o bastante.

O documento

Arquitetura em seis diagramas

O núcleo

AuthorizationCore — 16 degraus, ordem fixa

A invariante

Autoridade antes de dinheiro

Duas cores atravessam todos os diagramas. Azul é autoridade: quem pode. Âmbar é dinheiro: quanto. A ordem entre elas é a tese inteira.

As cores carregam significado, não decoração

|                  |                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| autoridade       | Quem pode. O mandato existe, não foi revogado, não expirou, o merchant e a categoria estão no escopo, o cartão é o que ele nomeia.        |
| dinheiro         | Quanto. Moeda e escala conferem, o valor é positivo, está abaixo do teto, há vaga de reserva, cabe na frequência e no orçamento.          |
| authorized       | Dentro do mandato. A compra segue para captura.                                                                                           |
| awaiting_human   | Fora do mandato, mas aprovável. Vira escalação com handle assinável. Não é falha: é o gatilho da decisão humana — e ela volta assinada.   |
| rejected         | Fora do mandato e não aprovável. Teto, revogação, expiração. Não há botão de aprovar, porque não há o que aprovar.                        |
| nunca consultado | O degrau que a avaliação não alcançou. A ausência é a evidência: uma revogação não pode ser contornada por uma compra pequena o bastante. |

Quatro perguntas, quatro provas — e elas não se substituem

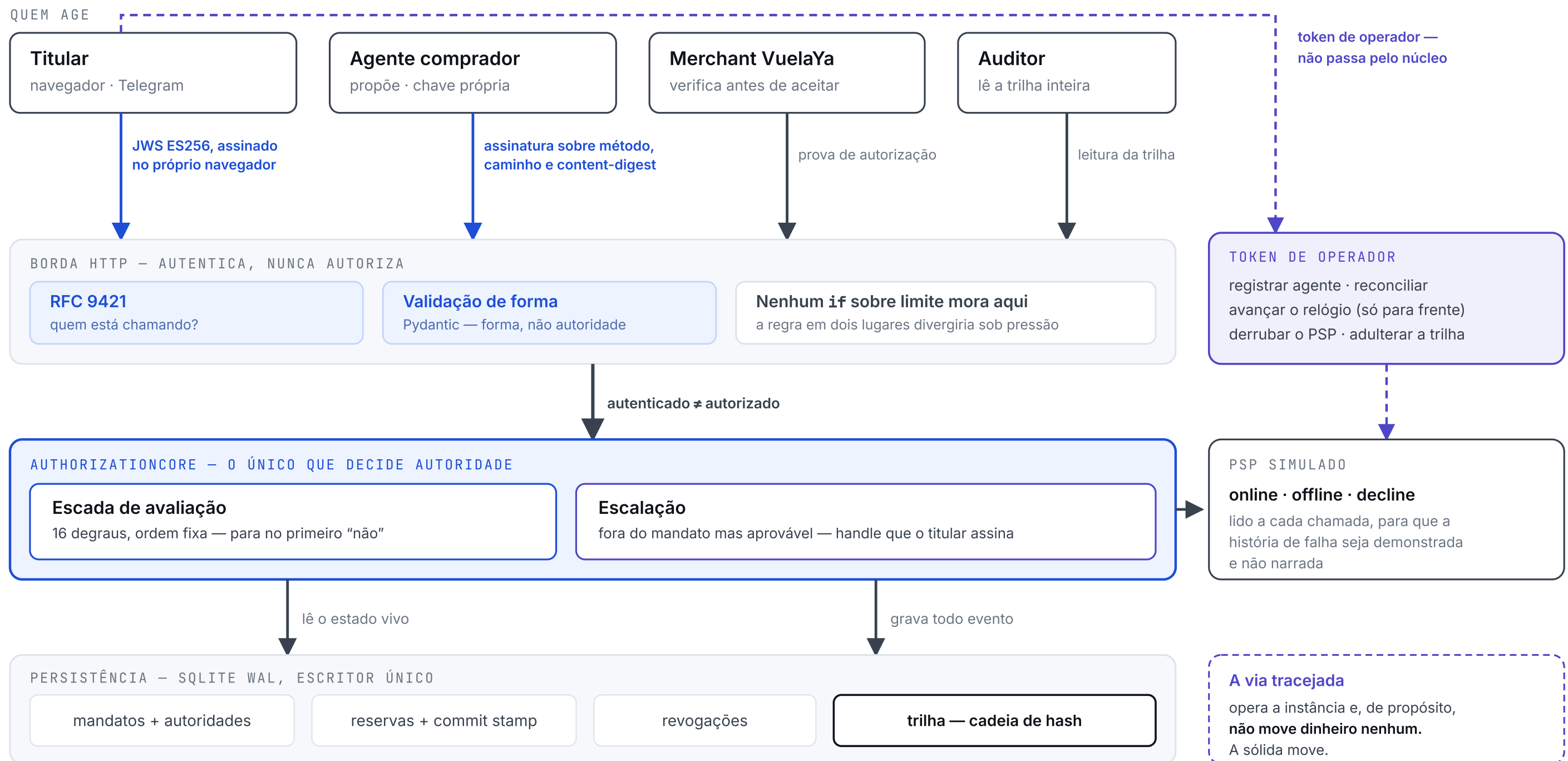
| PERGUNTA                         | PROVA                | QUEM DETÉM                     |
|----------------------------------|----------------------|--------------------------------|
| quem está chamando?              | assinatura RFC 9421  | o agente — chave própria       |
| esta compra pode acontecer?      | avaliação do mandato | ninguém: é determinística      |
| quem muda a autoridade de gasto? | JWS ES256            | o titular — chave no navegador |
| quem opera a instância?          | token de operador    | o time — e não move dinheiro   |

Autenticar não é autorizar. A borda HTTP responde a primeira pergunta e não tem um único if sobre limite: se a regra existisse em dois lugares, divergiria sob pressão.

|                                  |                                |                                      |                                        |
|----------------------------------|--------------------------------|--------------------------------------|----------------------------------------|
| 16                               | 3                              | 4                                    | 0                                      |
| degraus na escada, em ordem fixa | resultados possíveis, não dois | autoridades separadas, quatro provas | cartões no sistema: o PAN nunca existe |

## 02 Visão geral

Quatro atores, uma borda que só autentica, um núcleo que só decide. Repare nas duas setas que saem do titular.



A separação que mais importa está nas duas setas que saem do titular: uma leva uma **assinatura**, a outra leva um **token de operador**. A primeira move dinheiro; a segunda opera a instância — e nunca toca no núcleo. É por isso que um jurado pode derrubar o processador, avançar o relógio e adulterar a trilha **sem nunca conseguir aprovar uma compra**.

03

# A escada de avaliação

A ordem é a regra. O núcleo para no primeiro degrau que falha, e os degraus abaixo **nunca são consultados**.

Autoridade

quem pode

degraus 1-10

|    |                                        |                           |                |
|----|----------------------------------------|---------------------------|----------------|
| 1  | mandato existe?                        | mandate_not_found         | rejected       |
| 2  | revogação legível?                     | revocation_unavailable    | rejected       |
| 3  | não revogado?                          | mandate_revoked           | rejected       |
| 4  | merchant não revogado?                 | merchant_revoked          | rejected       |
| 5  | instrumento não revogado?              | instrument_revoked        | rejected       |
| 6  | orçamento não zerado?                  | budget_revoked            | awaiting_human |
| 7  | dentro da validade?                    | mandate_expired           | rejected       |
| 8  | merchant no escopo?                    | merchant_not_allowed      | awaiting_human |
| 9  | categoria no escopo?                   | category_not_allowed      | awaiting_human |
| 10 | é o cartão do mandato? — só na captura | instrument_not_in_mandate | rejected       |

AUTORIDADE PRIMEIRO

Dinheiro

quanto

degraus 11-16

|    |                          |                      |                |
|----|--------------------------|----------------------|----------------|
| 11 | moeda e escala conferem? |                      | rejected       |
| 12 | valor positivo?          |                      | rejected       |
| 13 | abaixo do teto?          | sem botão de aprovar | rejected       |
| 14 | há vaga de reserva?      | sem botão de aprovar | rejected       |
| 15 | dentro da frequência?    | usage_limit          | awaiting_human |
| 16 | dentro do orçamento?     | budget_exceeded      | awaiting_human |

→

authorized

os dezesseis responderam sim; a compra segue para captura

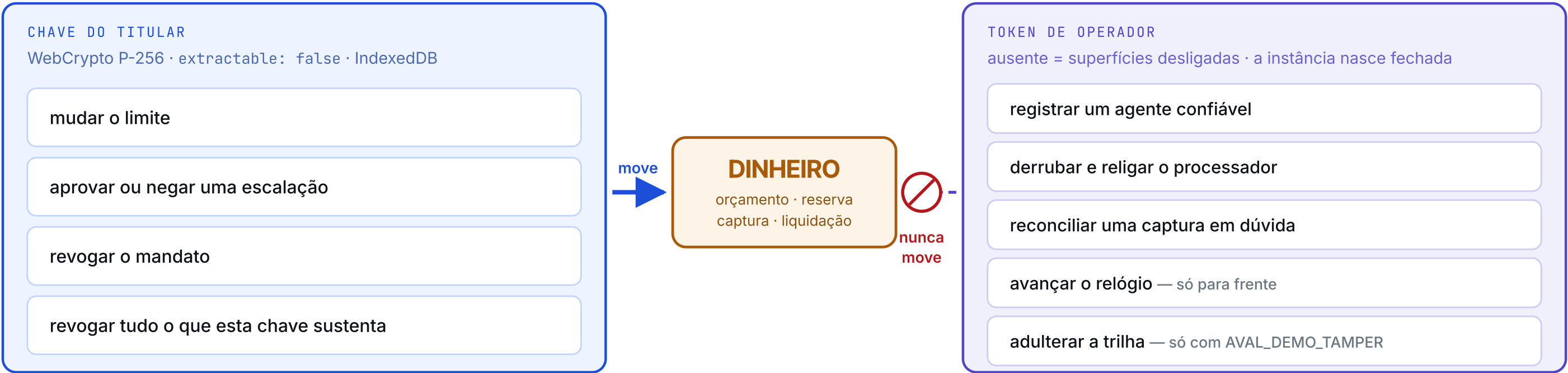
O **teto** e a **vaga de reserva** recusam em vez de escalar, de propósito: apertar aprovar não solta dinheiro que já está preso — quem solta é `POST /reconcile`.

## Um mandato revogado, avaliado degrau a degrau

A avaliação para no 3. Os treze seguintes **nunca são consultados** — e é a ausência deles que prova que uma revogação não pode ser contornada por uma compra pequena o bastante.

|          |          |          |        |        |        |        |        |        |         |         |         |         |         |         |         |
|----------|----------|----------|--------|--------|--------|--------|--------|--------|---------|---------|---------|---------|---------|---------|---------|
| 1<br>sim | 2<br>sim | 3<br>não | 4<br>— | 5<br>— | 6<br>— | 7<br>— | 8<br>— | 9<br>— | 10<br>— | 11<br>— | 12<br>— | 13<br>— | 14<br>— | 15<br>— | 16<br>— |
|----------|----------|----------|--------|--------|--------|--------|--------|--------|---------|---------|---------|---------|---------|---------|---------|

Um jurado opera o sistema com as duas. **Só uma delas chega ao dinheiro** — e isso é estrutural, não uma promessa de operação.



### O relógio só avança

Rebobiná-lo reviveria um mandato expirado — e isso seria um operador **devolvendo autoridade de gasto**. A alavanca existe para o jurado ver o mandato morrer na frente dele, não para ressuscitá-lo.

### A aprovação é evidência, não atalho

O JWS do titular nomeia `decision_handle`, `mandate_id`, `decision` e `amount`, os quatro conferidos contra a escalação congelada, e é guardado **inteiro** na trilha. Sem isso, uma aprovação poderia ser levantada de uma compra e aplicada a outra maior.

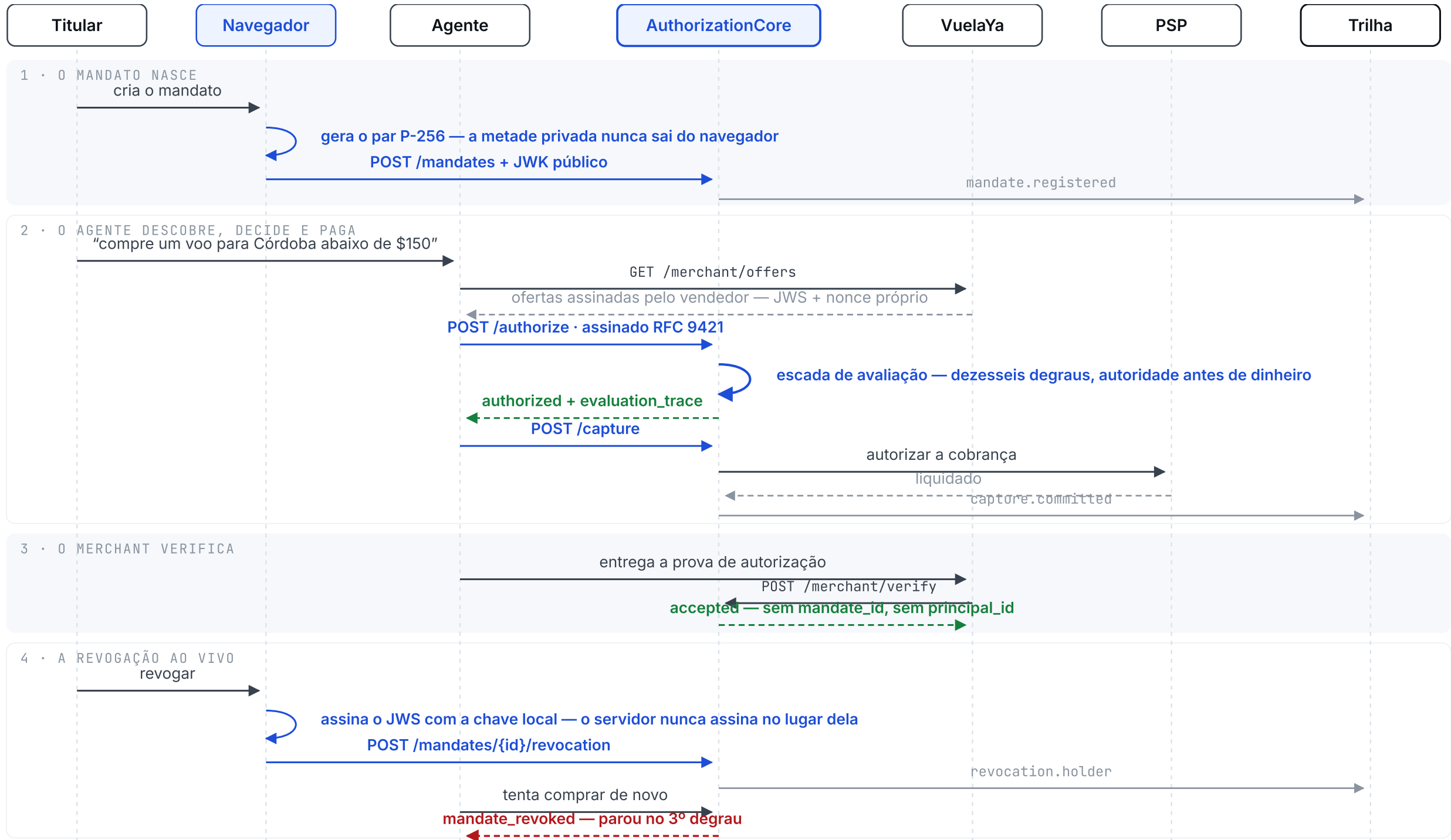
### E a aprovação não ressuscita nada

Tudo é reavaliado no momento da retomada: uma revogação que chegou **enquanto a pessoa decidia** ainda recusa a compra. O caminho de volta existe; ele apenas não passa por cima da escada.

05

# O circuito completo de uma compra

Mandato, compra, verificação e revogação — em quatro tempos, com a trilha recebendo cada um deles.



O merchant verifica sem saber quem comprou. A prova vincula checkout\_id, merchant\_id, valor, moeda e terms\_hash — e omite mandate\_id e principal\_id. Um recibo que vazasse o orçamento da compradora vazaria a compradora.

O agente em processo não tem privilégio. Ele assina e passa pela mesma verificação da borda HTTP. Rodar dentro do processo não compra confiança nenhuma — e é o que torna a defesa contra um agente impostor honesta em vez de acidental.





## Canonicalização, não formatação

Cada evento canonicaliza a si mesmo por **RFC 8785** antes de entrar no digest. Sem isso, reordenar chaves de um JSON mudaria o hash sem mudar o fato — e a cadeia acusaria fraude onde houve só serialização.

## O jurado quebra o elo, ao vivo

Com `AVAL_DEMO_TAMPER=1` existe uma rota que adultera um evento. Sem a variável ela **não é montada** — 404 de verdade, ausente até do OpenAPI. A instância nasce sem a alavanca destrutiva.

## Três projeções, uma verdade

Titular, merchant e auditor leem a **mesma** cadeia. A visão do merchant é montada por **lista branca** de eventos e de campos: uma lista negra esquece; uma lista branca não pode vaziar o campo que ninguém lembrou de esconder.



As dependências apontam para dentro. O núcleo não sabe o que foi comprado — e é isso que permite trocar o merchant sem tocar em uma linha de regra.





**SQLite, com WAL e `BEGIN IMMEDIATE`**

Escritor único e transação imediata, porque a corrida que importa — capturar enquanto alguém revoga — precisa de uma ordem, não de otimismo.

**A saída:** os repositórios estão atrás de portas. Trocar por Postgres não toca no núcleo.

**Custódia de chave em memória, no servidor**

Em produção seria HSM ou KMS, e a interface `KeyCustodyService` já é exatamente a que um HSM implementaria: chaves não cruzam a fronteira do serviço.

**E a que mais importa já saiu:** a chave do **titular** não vive no servidor — ela é do navegador, e nunca foi serializável.

**PSP simulado, e controlável de propósito**

Um processador que só funciona não demonstra nada. Este liga, cai e recusa sob comando, para que o **terceiro estado do pagamento** — em dúvida, com o orçamento retido — seja mostrado em vez de narrado. Timeout não é recusa, e não é sucesso.

**Sem PAN em lugar nenhum**

Não é que o cartão esteja bem guardado: **ele nunca existe no sistema**. O número é lido num único ponto, tokenizado na borda e descartado; o mandato guarda um token e quatro dígitos, e nenhum dos dois reconstrói um cartão.

**Nonces em processo, idempotência no banco**

O `ReplayGuard` é a camada barata na frente; a proteção **durável** contra cobrança dupla é a idempotência persistida, com janela de retenção e purga explícita.

Um nonce é queimado **por último**, depois de a assinatura verificar — senão um atacante gastaria nonces alheios com lixo assinado por ninguém.

**O agente decide por regras, e opcionalmente por modelo**

Sem chave de API, um clone limpo roda o case inteiro. Com chave, quem escolhe a oferta é um LLM — e **nada mais no sistema muda**: sem chave, com timeout, com JSON inválido ou com um SKU inventado, as regras assumem sozinhas.

É isso que torna a alucinação **demonstrável** em vez de narrada, e a demo independente de rede.

Nenhuma dessas fronteiras está no caminho de confiança. Todas podem ser trocadas sem que uma única regra sobre **quem pode gastar quanto** mude de lugar — que é a única propriedade que este desenho existe para proteger.